Java Bug-Fix Practice Drills

Oct 3, 2026 · @Vv

How to practice

Each drill gives a task, buggy code and the number of bugs to find; write your fixes before you open the Answers tab.

1. Set a timer: 5 minutes for warm-ups, 10 for harder drills, 25 for Karat-style rounds.
2. Read the task first, then the code, then run the 60-second checklist from the study guide.
3. Write each bug as one line: the input that breaks it, and what happens.
4. Write the fixed method, then trace one normal input and one edge case by hand.
5. Open the Answers tab. Score 1 point per bug found and 1 point for a working fix.
6. Redo any drill where you missed a bug three days later.

The real interview won't tell you how many bugs there are; the counts here are only for self-checking. Pattern numbers refer to Java Bug-Fix Patterns — Study Guide.

Model used in every drill

```
public class Payment {
    private String id;               // "P1"
    private String fromAccount;      // "ACC-1", the sender
    private String toAccount;        // "ACC-9", the receiver
    private double amount;           // 250.0
    private String currency;         // "GBP"
    private String status;           // "COMPLETED", "REJECTED", "PENDING"
    private int retryCount;          // 0 to 3
    private LocalDateTime createdAt;
    private List<String> labels;     // ["international", "urgent"], can be
null

    // constructor and getters omitted
}
```

Warm-up drills (5 minutes each)

D1. Account set

Task: active accounts are kept in a `HashSet`. After a deposit, the account must still be found in the set.

```
class Account {
    private final String accountNumber;
    private double balance;
    // constructor omitted

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Account)) return false;
        return accountNumber.equals(((Account) o).accountNumber);
    }

    @Override
    public int hashCode() {
        return Objects.hash(accountNumber, balance);
    }

    public void deposit(double amount) {
        balance += amount;
    }
}

Set<Account> active = new HashSet<>();
Account acc = new Account("ACC-1", 100);
active.add(acc);
acc.deposit(50);
active.contains(acc);   // expected: true
```

Bugs to find: 1, and give two reasons it is wrong. Study patterns: 1.

D2. Group payments by sender

Task: return every sender's payments, grouped by `fromAccount`.

```
Map<String, List<Payment>> bySender(List<Payment> payments) {
    Map<String, List<Payment>> result = new HashMap<>();
    for (Payment p : payments) {
        List<Payment> list = result.getOrDefault(p.getFromAccount(), new
ArrayList<>());
        list.add(p);
    }
    return result;
}
```

Bugs to find: 1. Then extend: return the number of payments per sender as a `Map<String, Integer>`. Study patterns: 2.

D3. Total per currency

Task: return the total amount of COMPLETED payments per currency, sorted by currency code.

```
Map<String, Double> totalByCurrency(List<Payment> payments) {
    return payments.stream()
            .collect(Collectors.toMap(Payment::getCurrency,
Payment::getAmount, (a, b) -> a));
}
```

Bugs to find: 3. Study patterns: 4, 5.

D4. High-value split

Task: count the payments of 10,000 or more, and the rest, for a daily report.

```
Map<Boolean, Long> counts = payments.stream()
        .collect(Collectors.groupingBy(p -> p.getAmount() > 10_000,
Collectors.counting()));

long highValue = counts.get(true);
long normal = counts.get(false);
```

Bugs to find: 2. Study patterns: 6, 17.

D5. Labels of completed payments

Task: return the distinct labels used on COMPLETED payments, sorted A to Z.

```
List<String> labelsOfCompleted(List<Payment> payments) {
    return payments.stream()
            .filter(p -> p.getStatus() == "COMPLETED")
            .flatMap(p -> p.getLabels().stream())
            .collect(Collectors.toSet())
            .stream()
            .collect(Collectors.toList());
}
```

Bugs to find: 3. Study patterns: 7, 9, 31.

D6. Average sent by an account

Task: return the average amount of COMPLETED payments sent from `account`, or 0.0 if there are none.

```
double averageSent(List<Payment> payments, String account) {
    double total = 0;
    int count = 0;
    for (Payment p : payments) {
        if (p.getFromAccount().equals(account) &&
p.getStatus().equals("COMPLETED")) {
            total += p.getAmount();
        }
        count++;
    }
    return total / count;
}
```

Bugs to find: 3. Hint: try ACC-1 sending 100 and 200, while ACC-2 sends 900. Study patterns: 11, 15.

D7. Oldest pending payment

Task: return the id of the oldest PENDING payment. If there is none, throw `IllegalStateException` with a clear message.

```
String oldestPendingId(List<Payment> payments) {
    return payments.stream()
            .filter(p -> "PENDING".equals(p.getStatus()))
            .max(Comparator.comparing(Payment::getCreatedAt))
            .get()
            .getId();
}
```

Bugs to find: 2. Study patterns: 12, 19.

D8. Payment queue sort

Task: sort the operations queue so PENDING payments come first and everything else after. Within each group, highest amount first; ties by id, A to Z.

```
payments.sort(Comparator.comparing(Payment::getStatus)
        .thenComparing(Payment::getAmount)
        .thenComparing(Payment::getId)
        .reversed());
```

Bugs to find: 2. Hint: sort P1 (COMPLETED, 100), P2 (PENDING, 500), P3 (PENDING, 100), P4 (REJECTED, 900), P5 (COMPLETED, 100) by hand. Study patterns: 13, 14.

Harder drills (10 minutes each)

D9. Urgent payment ids

Task: return the ids of payments labelled "urgent". The method must never return null.

```
List<String> urgentIds(List<Payment> payments) {
    if (payments == null || payments.isEmpty()) {
        return null;
    }
    List<String> ids = new ArrayList<>();
    for (Payment p : payments) {
        if (p.getLabels().contains("urgent")) {
            ids.add(p.getId());
        }
    }
    return ids;
}
```

Bugs to find: 3. Study patterns: 15, 16.

D10. Latest retry per payment id

Task: every retry creates a new row with the same id and a higher `retryCount`. Keep only the latest attempt for each id, in the order the ids first appeared.

```
List<Payment> latestAttempts(List<Payment> payments) {
    Map<String, Payment> byId = new HashMap<>();
    for (Payment p : payments) {
        byId.putIfAbsent(p.getId(), p);
    }
    return new ArrayList<>(byId.values());
}
```

Bugs to find: 2. Hint: input P7 (retry 0), P2, P7 (retry 1), P9, P7 (retry 2) should give P7 (retry 2), P2, P9. Study patterns: 5, 18, 24.

D11. Pagination

Task: return page `pageNumber` (pages start at 1) with `pageSize` items. A page past the end is an empty list. Invalid input throws `IllegalArgumentException`.

```
List<Payment> page(List<Payment> payments, int pageNumber, int pageSize) {
    int from = pageNumber * pageSize;
    int to = from + pageSize;
    return payments.subList(from, to);
}
```

Bugs to find: 4. Hint: with 5 payments and page size 2, pages 1, 2, 3 and 4 should return [P1, P2], [P3, P4], [P5] and []. Study patterns: 17, 22, 25.

D12. Retry eligibility

Task: a payment can be retried if its status is REJECTED or PENDING, and its `retryCount` is less than 3, and it was created within the last 24 hours.

```
boolean eligibleForRetry(Payment p, LocalDateTime now) {
    return p.getStatus().equals("REJECTED") ||
p.getStatus().equals("PENDING")
            && p.getRetryCount() <= 3
            && p.getCreatedAt().isAfter(now.minusHours(24));
}
```

Bugs to find: 3. Hint: what does it return for a REJECTED payment with 10 retries, created 5 days ago? Study patterns: 15, 17, 21.

D13. PaymentService

Task: the service owns its payment list. Callers can read the list, and can ask for the 5 biggest payments without changing anything.

```
class PaymentService {
    private final List<Payment> payments = new ArrayList<>();

    public List<Payment> getPayments() {
        return payments;
    }

    public List<Payment> biggestFive() {
        payments.sort((a, b) -> (int) (b.getAmount() - a.getAmount()));
        return payments.subList(0, 5);
    }
}
```

Bugs to find: 4. Study patterns: 25, 26, 22.

D14. Batch sending

Task: send every payment through the gateway. One failure must not stop the batch, and the caller needs to know which payments failed and why. `gateway.send()` throws a checked `GatewayException` on failure.

```
int processAll(List<Payment> payments, PaymentGateway gateway) {
    int processed = 0;
    try {
        for (Payment p : payments) {
            gateway.send(p);
            processed++;
        }
    } catch (Exception e) {
        System.out.println("error");
    }
    return processed;
}
```

Bugs to find: 3. Then extend: return a result object holding the succeeded ids and a map of failed id to reason. Study patterns: 29.

D15. Ledger reconciliation

Task: remove every payment whose amount doesn't match the ledger, then print "Reconciled" if the remaining count equals the expected count. Ledger amounts look like `new BigDecimal("250.00")`; a typical day has a few hundred payments.

```
void reconcile(List<Payment> payments, Map<String, BigDecimal> ledger,
Map<String, Integer> expected) {
    for (Payment p : payments) {
        BigDecimal ledgerAmount = ledger.get(p.getId());
        if (!ledgerAmount.equals(new BigDecimal(p.getAmount()))) {
            payments.remove(p);
        }
    }
    Integer remaining = payments.size();
    if (remaining == expected.get("count")) {
        System.out.println("Reconciled");
    }
}
```

Bugs to find: 5. Study patterns: 23, 31, 32, 33.

D16. Payments in a month

Task: return the payments created in a given year and month, for a monthly statement.

```
List<Payment> inMonth(List<Payment> payments, int year, int month) {
    LocalDateTime start = LocalDateTime.of(year, month, 1, 0, 0);
    LocalDateTime end = LocalDateTime.of(year, month, 30, 23, 59);
    return payments.stream()
            .filter(p -> p.getCreatedAt().isAfter(start) &&
p.getCreatedAt().isBefore(end))
            .collect(Collectors.toList());
}
```

Bugs to find: 4. Hint: try February, 31 October, 1 October at 00:00 and 30 October at 23:59:30. Study patterns: 22, 30.

Karat-style rounds (25 minutes each)

K1. StatementService

This mirrors the live round: existing code with bugs, then two new functions in the same class, then complexity and edge cases out loud.

Part 1: find and fix the bugs (aim for 10 minutes).

```
class StatementService {

    // Closing balance = opening + money received - money sent. COMPLETED
payments only.
```

```
    public double closingBalance(double opening, List<Payment> payments,
String account) {
        double balance = opening;
        for (Payment p : payments) {
            if (p.getStatus().equals("COMPLETED")) {
                if (p.getFromAccount().equals(account)) {
                    balance += p.getAmount();
                }
                if (p.getToAccount().equals(account)) {
                    balance -= p.getAmount();
                }
            }
        }
        return balance;
    }

    // The largest COMPLETED payment sent from the account.
    public Payment largestOutgoing(List<Payment> payments, String account) {
        Payment largest = payments.get(0);
        for (int i = 1; i <= payments.size(); i++) {
            Payment p = payments.get(i);
            if (p.getFromAccount().equals(account) && p.getAmount() >
largest.getAmount()) {
                largest = p;
            }
        }
        return largest;
    }
}
```

Bugs to find: 5 or more. Test data: opening 1,000 for ACC-1; ACC-1 sends 200 to ACC-2; ACC-3 sends 50 to ACC-1; ACC-2 sends 75 to ACC-1; ACC-1 sends 999 to ACC-4 but it was REJECTED. The correct closing balance is 925.

Part 2: add two methods (aim for 15 minutes).

1. `Map<String, Double> netFlowByCounterparty(List<Payment> payments, String account)`: for every other account, money received from it minus money sent to it, COMPLETED only, sorted by account. With the data above: {ACC-2=-125.0, ACC-3=50.0}.
2. `List<String> accountsWithRepeatedRejections(List<Payment> payments)`: sending accounts with 3 or more REJECTED payments on the same calendar day, sorted, each listed once.

Part 3: state the time complexity of each method and the edge cases you would test.

K2. LimitChecker

This round adds the questions a senior interviewer asks next: what changes when the clock moves, and when two requests arrive at once.

Part 1: find and fix the bugs (aim for 10 minutes).

```
class LimitChecker {
    private static final double DAILY_LIMIT = 5000;
    private final Map<String, Double> spentToday = new HashMap<>();

    // Approves and records the payment if it keeps the account within
today's limit.
    public boolean tryPay(String account, double amount) {
        double spent = spentToday.get(account);
        if (spent + amount < DAILY_LIMIT) {
            spentToday.put(account, spent + amount);
            return true;
        }
        return false;
    }
}
```

Bugs to find: 5. Hint: a payment that takes the total to exactly 5,000 is allowed, and this object lives for weeks inside a web service.

Part 2: add two methods (aim for 10 minutes).

1. `double remainingToday(String account)`: how much the account can still pay today.
2. `List<String> accountsNearLimit(double percent)`: accounts that have used at least `percent` of today's limit, highest usage first.

Part 3 (talk, don't code): the service now runs on 3 servers behind a load balancer. What breaks, and how would you fix it?